UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Protéger et surveiller en production

CL-SECU-API

StockLine a appris à dire « non ». Aujourd'hui, elle apprend à dire « pas si vite », puis on la place derrière une passerelle, dans un conteneur durci, et on la surveille.

Utopios® Tous droits réservés

Objectifs de la journée

À la fin de la journée, vous saurez :

- **Limiter** ce qu'une API consomme : pages plafonnées, valeurs bornées, tentatives de connexion comptées (API4).
- **Placer** une passerelle **nginx** devant l'API : HTTPS, quotas par adresse IP, filtrage, journal d'accès JSON.
- **Durcir** le conteneur de l'API : sans privilèges, en lecture seule, avec des plafonds de ressources.
- **Reconnaître** les risques API6 à API10 et citer, pour chacun, une protection.
- **Intégrer** les contrôles de sécurité à la chaîne de livraison : tests, analyse d'image, scan passif (DevSecOps).

Tout se fait **en conteneurs** (Docker Engine et Docker Compose) dans votre VM `srv-linux` : aucun compte en ligne n'est nécessaire.

Utopios® Tous droits réservés

Rappel du jour 1

StockLine a reçu quatre protections, écrites dans le code et **prouvées par des tests**.

Risque OWASP API 2023	La question	La mesure
API1 — Autorisation au niveau des objets	A-t-il droit à **cet** objet ?	chaque gestionnaire limité à son entrepôt ; un produit d'ailleurs répond 404, comme s'il n'existait pas
API2 — Authentification	Qui appelle ?	jeton JWT signé, valable 15 minutes ; secret Docker, hors du code et de l'image ; mots de passe hachés ; 401 sans jeton
API3 — Autorisation au niveau des propriétés	Quels **champs** entrent et sortent ?	schémas Pydantic en liste blanche : `prix_achat` ne sort pas, `entrepot` est fixé par le serveur
API5 — Autorisation au niveau des fonctions	A-t-il droit à **cette** action ?	suppression réservée au rôle `admin`, 403 pour les autres

Ces protections sont **nécessaires**, pas suffisantes : une API exposée peut aussi être **submergée**, **mal configurée** ou **oubliée**. C'est le programme d'aujourd'hui.

Utopios® Tous droits réservés

1. API4 — Consommation de ressources illimitée

Chaque requête a un coût

Utopios® Tous droits réservés

Chaque requête a un coût

Ressource	Ce qui la consomme	Quand elle s'épuise
Processeur	les calculs, le hachage d'un mot de passe (PBKDF2, 600 000 itérations)	l'API ralentit pour tout le monde
Mémoire	une liste entière chargée d'un coup	le processus est arrêté, les voisins souffrent
Bande passante	des réponses énormes	lenteur, coûts de transfert
Base de données	des lectures sans fin, des écritures en masse	saturation, temps de réponse
Argent 💰	en hébergement facturé à l'usage : le calcul et le transfert	une facture sans plafond

Ce que StockLine accepte au début de la derniere demo:

- `?limit=2` est **ignoré** : la liste entière revient ;
- une entrée de stock d'**un milliard** d'unités : `201 Created` ;
- 20 mauvais mots de passe d'affilée : 20 fois `401`, **jamais** de blocage.

Utopios® Tous droits réservés

Paginer et plafonner les valeurs

```python
# app/main.py : 20 par défaut, 100 au plus
limit: int = Query(20, ge=1, le=100)
offset: int = Query(0, ge=0)
...
requete.limit(limit).offset(offset)

# app/models.py : chaque nombre a un plafond
quantite: int = Field(gt=0, le=10_000)
prix_unitaire: float = Field(gt=0, le=100_000)
seuil_alerte: int = Field(default=5, ge=0, le=10_000)
```

- `ge`, `le` : « supérieur ou égal », « inférieur ou égal ».
- Hors limites : **422**, avant même d'interroger la base.
- La base ne lit que la page demandée.
- Un nombre sans plafond, c'est un stock d'un milliard, un seuil absurde, un prix délirant.

Pourquoi 100 au plus, si le client peut demander la page suivante avec `offset` ? Parce que chaque page devient une **requête** : elle repasse par l'authentification, les journaux… et les quotas. Le coût d'une requête reste **borné**.

Utopios® Tous droits réservés

Limiter les tentatives de connexion

Règle : après **5 échecs** en 5 minutes, le compte est bloqué 5 minutes, **même avec le bon mot de passe**.

```
# app/securite.py : appelé AVANT de vérifier le mot de passe
ECHECS_MAX = 5
FENETRE_ECHECS = timedelta(minutes=5)
...
if len(recents) >= ECHECS_MAX:
    raise HTTPException(status_code=status.HTTP_429_TOO_MANY_REQUESTS,
                        headers={"Retry-After": str(int(FENETRE_ECHECS.total_seconds()))})
```

```
Codes HTTP de 7 connexions de bruno avec un mauvais mot de passe : 401 401 401 401 401 429 429
OK : en-tête Retry-After (300)
OK : alice n'est pas bloquée (200)
```

- **429** *Too Many Requests* : « trop de requêtes, ralentissez ». Ni 401 (« qui êtes-vous ? »), ni 403 (« pas le droit »).
- `Retry-After: 300` : le nombre de secondes à attendre. Un client bien écrit patiente au lieu d'insister.
- **Compromis** : l'utilisateur qui se trompe 5 fois attend 5 minutes. Seul **ce** compte est bloqué.

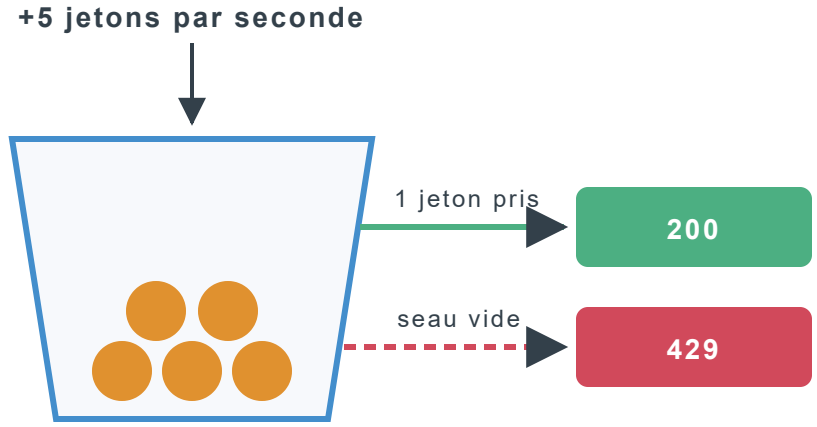
Utopios® Tous droits réservés

Compter les requêtes : deux algorithmes

Pour répondre 429 au bon moment, il faut **compter**. Deux façons classiques :

La fenêtre fixe. Un compteur par tranche de temps (« 5 requêtes par minute »), remis à zéro au début de chaque tranche. Simple à comprendre. *Défaut* : un client peut tout envoyer à la fin d'une tranche, puis au début de la suivante, soit le double en quelques secondes.

Le seau à jetons. Le seau garde quelques jetons d'avance (la **rafale**) et se remplit à un rythme fixe (le **débit**). Chaque requête prend un jeton ; seau vide : 429. Il tolère un pic court et impose un débit moyen.



les jetons d'avance : la rafale

La passerelle nginx de l'exercice 5 : 5 jetons par seconde et par adresse IP, rafale de 10

Le compteur de StockLine (exercice 4) regarde les échecs des **5 dernières minutes** : une fenêtre qui glisse avec le temps, sans effet de frontière.

Utopios® Tous droits réservés

Où poser les limites ?

Niveau	Exemple	Sa force	Sa limite
Le code	demo 4 : pages, plafonds, 5 échecs par compte	des règles **métier** : par compte, par champ	la requête a déjà coûté : elle a atteint l'API
La passerelle nginx	demo 5 : quotas par adresse IP, corps de 16 Ko au plus	arrête l'excès **avant** l'API, sans la solliciter	ne connaît ni les comptes ni les champs
Les plafonds du conteneur	demo 5 : 0,5 processeur, 256 Mo de mémoire, 64 processus	un emballement reste **confiné** : la VM et les autres conteneurs tiennent	ne refuse rien : l'API ralentit, ou est arrêtée puis relancée
Le pare-feu applicatif (option)	ModSecurity et les règles OWASP CRS, devant la passerelle	des règles communes à toutes les API, mises à jour	ne connaît pas la logique métier

⚠️ **Le piège du compteur en mémoire.** Celui de l'exercice 4 est propre à **chaque conteneur** : il disparaît au redémarrage, et 3 réplicas de l'API derrière un répartiteur de charge font 3 compteurs, donc jusqu'à 3 fois plus d'essais. En production : un compteur **partagé** (Redis) ou la limite de la **passerelle**, commune à tous les réplicas.

Les niveaux se complètent : c'est la **défense en profondeur**.

Utopios® Tous droits réservés

2. Passerelle et conteneur durci

Une porte unique, une API sans privilèges

Utopios® Tous droits réservés

Le rôle d'une passerelle

Une **passerelle** (ici **nginx**, en *reverse proxy*) est le **seul** point d'entrée : chaque requête la traverse avant d'atteindre l'API.

```
Client ──HTTPS :8443──> passerelle (nginx) ──réseau interne──> api (StockLine)
                        TLS, quotas par IP,                    non root, lecture seule,
                        taille, méthodes,                      0,5 CPU / 256 Mo / 64 processus,
                        en-têtes, journal JSON                 aucun port publié
```

Fonction	Ce qu'elle apporte	Dans la demo
Point d'entrée unique	une seule porte à surveiller et à inventorier	seul port publié : `127.0.0.1:8443` ; l'API n'en a plus
TLS	des échanges chiffrés, un serveur authentifié	TLS 1.2 et 1.3 ; certificat autosigné pour le labo
Quotas	répondre 429 **sans solliciter** l'API	par adresse IP : 5 requêtes par seconde, 10 connexions par minute
Filtrage	refuser ce que l'API n'a pas à recevoir	taille du corps, méthodes, documentation masquée
Journal	une trace de **chaque** requête, refus compris	une ligne JSON par requête

Sans passerelle, en test : 30 requêtes simultanées, 30 réponses `200`. Rien n'arrête un client qui en envoie des milliers.

Utopios® Tous droits réservés

Des quotas par adresse IP

```
# Une zone de comptage par usage ; la clé : l'adresse IP
limit_req_zone $binary_remote_addr zone=api:10m rate=5r/s;
limit_req_zone $binary_remote_addr zone=connexion:10m rate=10r/m;
limit_req_status 429;          # 503 par défaut

location = /auth/jeton {
    limit_req zone=connexion burst=5 nodelay;
    proxy_pass http://stockline;
}
location / {
    limit_req zone=api burst=10 nodelay;
    proxy_pass http://stockline;
}
```

- **Seau à jetons** : `rate` le remplit (5 jetons par seconde), `burst` lui donne une réserve (10). `nodelay` : la rafale passe tout de suite, sans file d'attente.
- **30 requêtes simultanées** : **11** réponses 200 (1 + la rafale de 10), **19** réponses 429. Le résultat se reproduit à l'identique.
- **8 connexions de suite**, avec le **bon** mot de passe : **6** acceptées (1 + 5), **2** refusées. Cette limite par adresse complète la limite **par compte** de l'exercice 4.
- Le quota est compté **par adresse IP** : un client insistant ne vide pas le seau des autres.

```
Codes HTTP pour 30 requêtes simultanées :
     11 200
     19 429
Codes HTTP pour 8 connexions de suite : 200 200 200 200 200 200 429 429
```

nginx parle de « seau qui fuit » (*leaky bucket*) ; avec `nodelay`, l'effet est celui d'un seau à jetons.

Utopios® Tous droits réservés

Filtrer, masquer, chiffrer

Mesure	Dans `passerelle/nginx.conf`	Effet vérifié	Risque
Taille des requêtes	`client_max_body_size 16k;`	corps de 20 Ko : **413**	API4
Méthodes autorisées	`if ($request_method !~ ^(GET\|POST\|DELETE)$) { return 405; }`	`PUT` : **405**	API8
Documentation masquée	`location ~ ^/(docs|redoc|openapi\.json) { return 404; }`	`/docs`, `/openapi.json` : **404**	API8, API9
Version cachée	`server_tokens off;`	`Server: nginx`, sans numéro	API8
En-têtes de sécurité	`add_header … always;`	HSTS, `nosniff`, CSP, `Cache-Control: no-store`	API8
HTTPS seul, TLS récent	`listen 8443 ssl;` · `ssl_protocols TLSv1.2 TLSv1.3;`	`/sante` en HTTPS : **200**	API8

Le certificat du labo est **autosigné** : `curl` le vérifie grâce à `--cacert`.

```
curl --cacert passerelle/certs/stockline.crt https://localhost:8443/sante
```

⚠️ **Jamais** `-k` (`--insecure`) : `curl` accepterait n'importe quel certificat, donc n'importe quel intermédiaire, et TLS ne protégerait plus rien. En production, le certificat vient d'une autorité reconnue (Let's Encrypt, autorité interne de l'entreprise) : les clients n'ont besoin d'aucune option.

Utopios® Tous droits réservés

Un conteneur durci

```dockerfile
# Dockerfile en trois étapes
FROM python:3.12-slim AS base
RUN pip install --no-cache-dir -r requirements.txt \
 && useradd --uid 10001 --no-create-home \
      --shell /usr/sbin/nologin stockline

# tests : pytest et les tests, jamais déployée
FROM base AS tests
RUN pip install --no-cache-dir -r requirements-dev.txt

# production : le seul dossier app/
FROM base AS production
COPY app ./app
USER stockline
CMD ["uvicorn", "app.main:app", …, "--no-server-header"]
```

```yaml
# compose.yaml, service api
api:
  build: { context: ., target: production }
  networks: [interne]          # aucun port publié
  read_only: true              # lecture seule…
  tmpfs: [/tmp]                # …sauf /tmp, en mémoire
  cap_drop: [ALL]              # aucune capacité Linux
  security_opt: ["no-new-privileges:true"]
  deploy:
    resources:
      limits: { cpus: "0.50", memory: 256M, pids: 64 }
```

Si une faille donnait à un attaquant l'exécution de code dans le conteneur :

- **`stockline`, UID 10001** : aucun droit d'administration dans le conteneur ;
- **`read_only: true`** : impossible de modifier le code ou de déposer un outil ; seuls `/tmp` (en mémoire) et le volume de données restent inscriptibles ;
- **`cap_drop: [ALL]`** et **`no-new-privileges`** : impossible de regagner des privilèges ;
- **plafonds** : un emballement ne sature ni la VM ni les autres conteneurs ;
- **étape `production`** : ni pytest ni les tests dans l'image, qui passe de 196 Mo à 182 Mo.

Utopios® Tous droits réservés

Le secret et le journal d'accès

Le secret de signature n'est lisible que par l'utilisateur de l'API :

```
sudo chown 10001:10001 secrets/jwt_secret.txt
sudo chmod 400 secrets/jwt_secret.txt
```

- Docker le monte dans `/run/secrets/jwt_secret` ; l'API n'en connaît que le **chemin**.
- `400` : lecture seule, pour le seul UID 10001, l'utilisateur `stockline`.
- `.dockerignore` exclut `secrets/` et `passerelle/` : ni le secret ni la clé privée TLS n'entrent dans l'image.

Le journal d'accès : une ligne JSON par requête, interrogée avec `jq`.

```
docker compose logs --no-log-prefix passerelle | grep '^{' \
  | jq -c 'select(.statut == "429")' | tail -1
```


```
{"date":"2026-10-30T09:12:41+00:00","ip":"172.19.0.1","methode":"POST",
 "chemin":"/auth/jeton","statut":"429","octets":"162",
 "duree_s":"0.000","api_s":""}
```


`api_s`, le temps de réponse de l'API, est **vide** : la requête refusée n'a **jamais atteint l'API**. La passerelle a absorbé l'excès.

Le chemin journalisé est `$uri`, sans la chaîne de requête (`?…`) : moins de données inutiles dans le journal.

⚠️ **RGPD** : l'adresse IP est une **donnée personnelle**. Durée de conservation limitée et écrite (la CNIL recommande de 6 mois à 1 an pour des journaux de sécurité), accès restreint, et jamais de jeton ni de mot de passe dans un journal.

Utopios® Tous droits réservés

3. API6 à API10 (survol)

Cinq risques à reconnaître, cinq réflexes à avoir

Utopios® Tous droits réservés

API6 — Accès illimité aux flux métier sensibles

Ce que c'est : un parcours métier **légitime**, utilisé de façon **automatisée** et massive. Chaque requête est valide : ce sont le **volume** et le **rythme** qui nuisent à l'entreprise ou aux autres clients. L'API a été pensée pour un humain qui clique, pas pour un script qui enchaîne.

Flux sensible	Usage automatisé	Conséquence
Achat de billets, de produits en quantité limitée	des robots achètent tout en quelques secondes	stock épuisé, revente au marché parallèle
Inscription	des milliers de faux comptes	spam, fraude aux offres de bienvenue
Parrainage, bons de réduction	des parrainages fictifs en série	des crédits distribués à tort

Se protéger : repérer les flux sensibles **avec le métier** ; limiter par compte, par appareil, par moyen de paiement (« deux billets par personne ») ; détecter l'automatisation (rythme non humain, adresses d'hébergeurs) ; ajouter une friction ciblée (CAPTCHA, confirmation par e-mail).

Vérifier : dans le journal d'accès, combien d'achats par compte, par adresse, par minute ?

Utopios® Tous droits réservés

API7 — Falsification de requêtes côté serveur (SSRF)

Ce que c'est : l'API va chercher une ressource à une **adresse fournie par le client** (aperçu de lien, import d'image, *webhook*). Sans contrôle, le serveur peut être amené à appeler une destination **interne**, que lui seul peut joindre.

Pourquoi c'est grave : le conteneur de l'API voit ce que le client ne voit pas. Les cibles internes sont les **autres services** de son réseau : un conteneur de base de données, une interface d'administration laissée sans mot de passe « puisqu'elle est interne », `localhost`, les adresses privées (`10.0.0.0/8`, `172.16.0.0/12` où vivent les réseaux Docker, `192.168.0.0/16`).

Protection	Principe
Liste blanche de destinations	n'appeler que les domaines prévus ; tout le reste est refusé
Blocage des adresses internes	refuser `localhost`, les adresses privées et réservées, **après** résolution du nom et à **chaque** redirection : un nom anodin peut désigner une adresse interne
Isolement réseau	des réseaux Docker séparés : le service qui appelle l'extérieur ne partage aucun réseau avec la base ni avec l'administration
Moindre privilège	chaque service interne exige sa propre authentification : être joignable ne suffit pas

Vérifier : avec `docker network inspect`, quels services partagent le réseau d'un conteneur qui appelle des adresses externes ?

Utopios® Tous droits réservés

API8 — Mauvaise configuration de sécurité

Ce que c'est : des réglages par défaut, de développement ou oubliés, laissés en production. Aucune ligne de code n'est fausse : c'est la **configuration** qui expose. L'exercice 5 en a corrigé la plupart.

Réglage à revoir	Le risque	Dans l'exercice 5
HTTP seul, TLS ancien	des échanges lisibles en transit	HTTPS seul, TLS 1.2 et 1.3
En-têtes de sécurité absents	le navigateur n'est pas aidé	HSTS, `nosniff`, CSP, `no-store`
Méthodes HTTP inutiles	une surface d'attaque élargie	`GET`, `POST`, `DELETE` seulement, 405 sinon
`/docs` et `/openapi.json` publics	la carte complète de l'API, offerte à tous	masqués par la passerelle : 404
En-tête `Server` bavard	la technologie et sa version révélées	`server_tokens off`, `--no-server-header`
Conteneur en `root`, sans plafond	une faille donne tous les droits, et toute la machine	`stockline`, lecture seule, `cap_drop`, plafonds
CORS `*`, erreurs détaillées	tout site peut appeler l'API ; des traces de pile divulguées	à régler dans le code

```
app = FastAPI(docs_url=None, redoc_url=None, openapi_url=None)  # en production, en plus de la passerelle
```

Vérifier : Trivy relit le `Dockerfile` et le `compose.yaml`, OWASP ZAP observe les réponses de la passerelle (démo).

Utopios® Tous droits réservés

API9 — Gestion défaillante de l'inventaire

Ce que c'est : on ne protège pas ce qu'on ignore. Une ancienne version restée en ligne à côté de la nouvelle, une route de test ou de débogage oubliée, une documentation qui ne correspond plus au code, un conteneur qui tourne encore sur une vieille image.

Pourquoi c'est grave : l'ancienne version garde ses **anciennes failles**. On corrige `/v2`… et `/v1` répond toujours.

Bonne pratique	Concrètement
Un **inventaire** tenu à jour	pour chaque API : propriétaire, environnement, version, données traitées, exposition
L'**inventaire des routes**	lister ce que l'application expose **vraiment** et le comparer à la documentation (démo, contrôle 4)
Les **images et versions** en service	quelle image, construite quand, tourne où ? Une image jamais reconstruite garde ses vulnérabilités
Des **étiquettes**	`labels:` dans `compose.yaml` : propriétaire, date de fin
Une **fin de vie** planifiée	l'ancienne version est annoncée, puis **supprimée**, conteneurs et images compris
Le **nettoyage** en fin de projet	ce module se termine par un nettoyage obligatoire

Vérifier : la démo liste les routes de StockLine. `/docs`, `/redoc` et `/openapi.json` y figurent toujours : c'est la **passerelle** qui les masque.

Utopios® Tous droits réservés

API10 — Consommation non sûre d'API tierces

Ce que c'est : votre API fait confiance à ce qui vient d'un **tiers** : les réponses des API qu'elle appelle (paiement, transporteur, géolocalisation…) et, par le même réflexe, les **dépendances** qu'elle embarque (bibliothèques Python, image de base). Compromis, vulnérable ou lent, le tiers devient **votre** problème.

Protection	Pourquoi
Traiter chaque réponse comme une **entrée non fiable**	la valider par un schéma strict, comme un corps de requête (exercice 3)
Des **délais d'attente** (*timeouts*), des essais bornés	un tiers lent ne doit pas immobiliser l'API ni accumuler les requêtes en attente
TLS vérifié, toujours	jamais `verify=False`, pas plus que `curl -k`
Une **taille** de réponse limitée, pas de redirection suivie à l'aveugle	lien avec API4 et API7
Des **dépendances** et une **image de base** analysées	Trivy à chaque construction : les 54 vulnérabilités hautes ou critiques de l'image de StockLine viennent de `python:3.12-slim` ; on reconstruit souvent pour récupérer les correctifs

```
reponse = requests.get(URL_TRANSPORTEUR, timeout=5)        # sans timeout, requests attend indéfiniment
tarif = TarifTransporteur.model_validate(reponse.json())   # schéma strict, sinon erreur
```

Utopios® Tous droits réservés

4. Tester la sécurité en continu

DevSecOps : chaque modification est vérifiée

Utopios® Tous droits réservés

La sécurité dans la chaîne de livraison

Les contrôles des exercices et de la démo ne protègent que s'ils tournent **à chaque modification**. Le **DevSecOps** les place dans l'intégration continue (CI), qui les lance **à chaque commit** : une régression de sécurité **bloque** la livraison, comme un bug.



Un contrôle en échec : la chaîne s'arrête, rien n'est livré.

Étape	Outil	Ce qu'elle vérifie
Tests de sécurité	pytest : les tests des exercices	les règles attendues : 401, 403, 404, 422, 429
Dépendances et image	Trivy `image`	les bibliothèques et les paquets à vulnérabilité connue (CVE)
Configuration	Trivy `config`	le `Dockerfile` et le `compose.yaml` : `root`, privilèges, ports
Scan passif	ZAP *baseline*, sur un environnement de **test**	en-têtes, fuites d'information, documentation exposée
Revue de code, à chaque demande de fusion	un relecteur, aidé d'une analyse statique (Bandit, Semgrep)	ce qu'un outil ne voit pas : la logique d'autorisation

⚠️ **On ne teste que ce qu'on est autorisé à tester** : ses propres systèmes, ou ceux couverts par une **autorisation écrite**. Tester le système d'un tiers sans son accord est une infraction pénale (art. 323-1 et suivants du Code pénal).

Utopios® Tous droits réservés

Nettoyage obligatoire

Une API oubliée qui tourne reste une porte ouverte : c'est exactement le risque API9. On supprime **tout** ce que le module a créé dans Docker, et **uniquement** cela.

```
docker compose --profile tests down -v --rmi local --remove-orphans
sudo rm -f secrets/jwt_secret.txt      # le secret de signature
rm -rf passerelle/certs                # le certificat et la clé privée TLS
```

Option	Ce qu'elle supprime, en plus des conteneurs et du réseau
`--profile tests`	le service `tests`, ignoré sans cette option
`-v`	le volume de données
`--rmi local`	les images construites pour le module, pas les images de base téléchargées
`--remove-orphans`	les conteneurs d'anciennes versions de `compose.yaml`

```
OK : plus aucun conteneur, volume, image, secret ni clé du module
```

- **Rien n'a été créé hors de la VM** : aucun compte, aucune ressource en ligne à supprimer.
- Le dossier `~/stockline-api` reste : c'est votre travail.

Utopios® Tous droits réservés

Synthèse des deux jours

Risque OWASP API 2023	La mesure vue	Où
API1 — Autorisation au niveau des objets	chaque gestionnaire limité à son entrepôt, 404 sinon	demo 2
API2 — Authentification	jeton JWT signé et court, secret Docker hors du code et de l'image, mots de passe hachés	demo 1
API3 — Autorisation au niveau des propriétés	schémas d'entrée et de sortie en liste blanche	demo 3
API4 — Consommation de ressources	pagination, plafonds, 5 échecs puis 429 ; quotas par IP à la passerelle ; plafonds du conteneur	demo 4 et 5
API5 — Autorisation au niveau des fonctions	suppression réservée à l'admin, 403 sinon	demo 2
API6 — Flux métier sensibles	limites par compte, détection de l'automatisation	Survol
API7 — SSRF	liste blanche, adresses internes bloquées, réseaux séparés	Survol
API8 — Mauvaise configuration	HTTPS, en-têtes, méthodes, documentation masquée, conteneur durci	demo 5
API9 — Inventaire	inventaire des routes exposées, images suivies, nettoyage	Démo
API10 — API tierces	réponses validées, délais d'attente, TLS vérifié ; image analysée	Survol et démo

Et pour que ça dure : tests, Trivy et ZAP **à chaque commit**.

Utopios® Tous droits réservés

Fin du module

Merci de votre attention

StockLine est passée de « répond à tout le monde » à une API **authentifiée**, **cloisonnée**, **limitée**, placée derrière une **passerelle**, dans un **conteneur durci**, et **surveillée**.

Utopios® Tous droits réservés